Додаток 1. Лістинг коду програми

apps/lab3.cpp

```cpp
#include <AbstractSyntaxTree.hpp>
#include <Args.hpp>
#include <CodeGenerator.hpp>
#include <DeclarationsTableView.hpp>
#include <FileSource.hpp>
#include <IdentifiersTableView.hpp>
#include <LexicalError.hpp>
#include <LiteralsTableView.hpp>
#include <Logger.hpp>
#include <Parser.hpp>
#include <SemanticAnalyzer.hpp>
#include <SemanticError.hpp>
#include <SymbolStore.hpp>
#include <SyntaxError.hpp>
#include <SyntaxTreeView.hpp>
#include <Tokenizer.hpp>
#include <TokensTableView.hpp>
#include <fstream>
#include <iostream>

int main(int argc, char* argv[]) {
    Args args(argc, argv);
    args.expectString("source", "s", true)
        .expectString("output", "o", false)
        .expectFlag("tokens", "t")
        .expectFlag("tree", "T")
        .expectFlag("identifiers", "i")
        .expectFlag("literals", "l")
        .expectFlag("declarations", "d")
        .expectFlag("asm", "a")
        .parse();

    auto errorFormatter = [](const auto& err) {
        return std::format(
            "Error [{}:{}]: {}", err.row + 1, err.column + 1, err.message
        );
    };

    SymbolStore symbols;
    CharacterAttributes attributes;

    FileSource source(args.getString("source"));
    Logger<LexicalError> tokenizerLogger("Tokenizer", errorFormatter);
    Tokenizer tokenizer(source, symbols, attributes, tokenizerLogger);
    tokenizer.scan();

    Logger<SyntaxError> parserLogger("Parser", errorFormatter);
    Parser parser(symbols, tokenizer.tokens(), parserLogger);
    parser.parse();

    AbstractSyntaxTree ast(parser.tree(), symbols);

    Logger<SemanticError> analyzerLogger("Semantics", errorFormatter);
    SemanticAnalyzer semantics(ast, analyzerLogger);
    semantics.analyze();

    if (args.getFlag("tokens")) {
        TokensTableView("\nTokens", symbols, tokenizer.tokens()).print();
    }

```

```
    if (args.getFlag("identifiers")) {
        IdentifiersTableView("\nIdentifiers", symbols.identifiers()).print();
    }

    if (args.getFlag("literals")) {
        LiteralsTableView("\nLiterals", symbols.literals()).print();
    }

    if (args.getFlag("tree")) {
        SyntaxTreeView("\nSyntax Tree", parser.tree(), symbols).print();
    }

    if (args.getFlag("declarations")) {
        DeclarationsTableView("\nDeclarations", semantics.declarations())
            .print();
    }

    const bool hasErrors = !tokenizerLogger.messages().empty() ||
                           !parserLogger.messages().empty() ||
                           !analyzerLogger.messages().empty();

    if (!hasErrors) {
        AssemblyFormatter assemblyFormatter({
            .labelWidth = 8,
            .instructionWidth = 8,
            .operandsWidth = 16,
        });

        CodeGenerator codegen(ast, assemblyFormatter);
        codegen.generate();

        const auto outputPath = args.getString("output");
        if (!outputPath.empty()) {
            std::ofstream file(outputPath);
            file << codegen.output();
            std::cout << "Assembly written to: " << outputPath << "\n";
        }

        if (args.getFlag("asm")) {
            std::cout << "\n" << codegen.output();
        }
    }

    return 0;
}
```

src/ast/data/AbstractSyntaxTree/AbstractSyntaxTree.hpp

```hpp
#pragma once

#include <AstNode.hpp>
#include <Declaration.hpp>
#include <SymbolStore.hpp>
#include <SyntaxData.hpp>
#include <Tree.hpp>
#include <TreeNode.hpp>
#include <memory>

class AbstractSyntaxTree {
private:
    const SymbolStore& _symbols;

    void foldAxiom(const TreeNode<SyntaxData>& node);
    std::unique_ptr<ProgramNode> foldProgram(const TreeNode<SyntaxData>& node);
    void foldBlock(const TreeNode<SyntaxData>& node, ProgramNode& program);
    void foldDeclarations(
        const TreeNode<SyntaxData>& node,
        ProgramNode& program
    );
    void foldConstantDeclarationsList(
        const TreeNode<SyntaxData>& node,
        ProgramNode& program
    );
    std::unique_ptr<ConstantNode> foldConstantDeclaration(
        const TreeNode<SyntaxData>& node
    );
    Value evaluateConstant(const TreeNode<SyntaxData>& node);
    Value evaluateComplexNumber(const TreeNode<SyntaxData>& node);

public:
    std::unique_ptr<RootNode> root;

    AbstractSyntaxTree(
        const Tree<SyntaxData>& tree, const SymbolStore& symbols
    );
    void accept(class AstVisitor& visitor) const;
};
```

src/ast/data/AbstractSyntaxTree/AbstractSyntaxTree.cpp

```cpp
#include "AbstractSyntaxTree.hpp"

#include <AstNode.hpp>
#include <AstVisitor.hpp>
#include <Rules.hpp>
#include <complex>
#include <memory>
#include <optional>
#include <string>

AbstractSyntaxTree::AbstractSyntaxTree(
    const Tree<SyntaxData>& tree,
    const SymbolStore& symbols
) :
    _symbols(symbols), root(std::make_unique<RootNode>()) {
    foldAxiom(tree.root());
}

void AbstractSyntaxTree::accept(AstVisitor& visitor) const {
    root->accept(visitor);
}

void AbstractSyntaxTree::foldAxiom(const TreeNode<SyntaxData>& node) {
    const auto& signalProgram = *node.children()[0];
    const auto& program = *signalProgram.children()[0];

    root->setProgram(foldProgram(program));
}

std::unique_ptr<ProgramNode> AbstractSyntaxTree::foldProgram(
    const TreeNode<SyntaxData>& node
) {
    const auto& procedureIdentifier = *node.children()[0];
    const auto& identifierTerminal = *procedureIdentifier.children()[0];
    const auto& identifierToken = *identifierTerminal.data().token;

    auto program = std::make_unique<ProgramNode>(
        _symbols.lookup(identifierToken.code), identifierToken.row,
        identifierToken.column
    );

    const auto& block = *node.children()[1];
    foldBlock(block, *program);

    return program;
}

void AbstractSyntaxTree::foldBlock(
    const TreeNode<SyntaxData>& node,
    ProgramNode& program
) {
    const auto& declarations = *node.children()[0];

    foldDeclarations(declarations, program);
    program.setStatements(std::make_unique<StatementsNode>());
}

void AbstractSyntaxTree::foldDeclarations(
    const TreeNode<SyntaxData>& node,
    ProgramNode& program
) {
    const auto& constantDeclarations = *node.children()[0];
```

```
63
64      if (constantDeclarations.data().rule ==
65          RuleKey::ConstantDeclarationsEmpty) {
66          return;
67      }
68
69      const auto& list = *constantDeclarations.children()[0];
70      foldConstantDeclarationsList(list, program);
71  }
72
73  void AbstractSyntaxTree::foldConstantDeclarationsList(
74      const TreeNode<SyntaxData>& node,
75      ProgramNode& program
76  ) {
77      if (node.data().rule == RuleKey::ConstantDeclarationsListEmpty) {
78          return;
79      }
80
81      const auto& declaration = *node.children()[0];
82      program.addConstant(foldConstantDeclaration(declaration));
83
84      const auto& rest = *node.children()[1];
85      foldConstantDeclarationsList(rest, program);
86  }
87
88  std::unique_ptr<ConstantNode> AbstractSyntaxTree::foldConstantDeclaration(
89      const TreeNode<SyntaxData>& node
90  ) {
91      const auto& constantIdentifier = *node.children()[0];
92      const auto& identifierTerminal = *constantIdentifier.children()[0];
93      const auto& identifierToken = *identifierTerminal.data().token;
94
95      const auto& constantValue = *node.children()[1];
96
97      return std::make_unique<ConstantNode>(
98          _symbols.lookup(identifierToken.code), evaluateConstant(constantValue),
99          identifierToken.row, identifierToken.column
100     );
101 }
102
103 Value AbstractSyntaxTree::evaluateConstant(const TreeNode<SyntaxData>& node) {
104     const auto& complexNumber = *node.children()[0];
105
106     return evaluateComplexNumber(complexNumber);
107 }
108
109 Value AbstractSyntaxTree::evaluateComplexNumber(
110     const TreeNode<SyntaxData>& node
111 ) {
112     const auto& leftPart = *node.children()[0];
113     const auto& rightPart = *node.children()[1];
114
115     std::optional<std::string> leftPartStr;
116     if (!leftPart.children().empty()) {
117         leftPartStr =
118             _symbols.lookup(leftPart.children()[0]->data().token->code);
119     }
120
121     std::optional<std::string> rightPartStr;
122     if (!rightPart.children().empty()) {
123         rightPartStr =
124             _symbols.lookup(rightPart.children()[0]->data().token->code);
125     }
126
127     if (rightPart.data().rule == RuleKey::RightPartExp) {
128         auto magnitude = std::stof(leftPartStr.value_or("1"));
129         auto exponent = std::stof(rightPartStr.value_or("0"));
```

```
130
131         return Value{std::polar(magnitude, exponent)};
132     }
133
134     auto real = std::stoi(leftPartStr.value_or("0"));
135     auto imaginary = std::stoi(rightPartStr.value_or("0"));
136
137     return Value{std::complex<int>{real, imaginary}};
138 }
```

src/ast/data/AstNode/AstNode.hpp

```hpp
#pragma once

#include <AstVisitor.hpp>
#include <Declaration.hpp>
#include <cstddef>
#include <memory>
#include <string>
#include <vector>

class AstNode {
public:
    virtual ~AstNode() = default;
    virtual void accept(AstVisitor& visitor) const = 0;
};

class ConstantNode : public AstNode {
private:
    std::string _identifier;
    Value _value;
    size_t _row;
    size_t _column;

public:
    ConstantNode(
        std::string identifier, Value value, size_t row, size_t column
    );

    [[nodiscard]] const std::string& identifier() const;
    [[nodiscard]] const Value& value() const;
    [[nodiscard]] size_t row() const;
    [[nodiscard]] size_t column() const;

    void accept(AstVisitor& visitor) const override;
};

class StatementsNode : public AstNode {
public:
    void accept(AstVisitor& visitor) const override;
};

class ProgramNode : public AstNode {
private:
    std::string _identifier;
    std::vector<std::unique_ptr<ConstantNode>> _constants;
    std::unique_ptr<StatementsNode> _statements;
    size_t _row;
    size_t _column;

public:
    ProgramNode(std::string identifier, size_t row, size_t column);

    [[nodiscard]] const std::string& identifier() const;
    [[nodiscard]] const std::vector<std::unique_ptr<ConstantNode>>& constants(
    ) const;
    [[nodiscard]] const StatementsNode* statements() const;
    [[nodiscard]] size_t row() const;
    [[nodiscard]] size_t column() const;

    void addConstant(std::unique_ptr<ConstantNode> constant);
    void setStatements(std::unique_ptr<StatementsNode> statements);

    void accept(AstVisitor& visitor) const override;
```

```
63 };
64
65 class RootNode : public AstNode {
66 private:
67     std::unique_ptr<ProgramNode> _program;
68
69 public:
70     [[nodiscard]] const ProgramNode* program() const;
71
72     void setProgram(std::unique_ptr<ProgramNode> program);
73
74     void accept(AstVisitor& visitor) const override;
75 };
```

src/ast/data/AstNode/AstNode.cpp

```cpp
#include "AstNode.hpp"

#include <utility>

// --- ConstantNode ---

ConstantNode::ConstantNode(
    std::string identifier,
    Value value,
    size_t row,
    size_t column
) :
    _identifier(std::move(identifier)),
    _value(std::move(value)),
    _row(row),
    _column(column) {}

const std::string& ConstantNode::identifier() const {
    return _identifier;
}

const Value& ConstantNode::value() const {
    return _value;
}

size_t ConstantNode::row() const {
    return _row;
}

size_t ConstantNode::column() const {
    return _column;
}

void ConstantNode::accept(AstVisitor& visitor) const {
    visitor.visitConstantNode(*this);
}

// --- StatementsNode ---

void StatementsNode::accept(AstVisitor& visitor) const {
    visitor.visitStatementsNode(*this);
}

// --- ProgramNode ---

ProgramNode::ProgramNode(std::string identifier, size_t row, size_t column) :
    _identifier(std::move(identifier)), _row(row), _column(column) {}

const std::string& ProgramNode::identifier() const {
    return _identifier;
}

const std::vector<std::unique_ptr<ConstantNode>>& ProgramNode::constants(
) const {
    return _constants;
}

const StatementsNode* ProgramNode::statements() const {
    return _statements.get();
}

size_t ProgramNode::row() const {
```

```cpp
    return _row;
}

size_t ProgramNode::column() const {
    return _column;
}

void ProgramNode::addConstant(std::unique_ptr<ConstantNode> constant) {
    _constants.push_back(std::move(constant));
}

void ProgramNode::setStatements(std::unique_ptr<StatementsNode> statements) {
    _statements = std::move(statements);
}

void ProgramNode::accept(AstVisitor& visitor) const {
    visitor.visitProgramNode(*this);
}

// --- RootNode ---

const ProgramNode* RootNode::program() const {
    return _program.get();
}

void RootNode::setProgram(std::unique_ptr<ProgramNode> program) {
    _program = std::move(program);
}

void RootNode::accept(AstVisitor& visitor) const {
    visitor.visitRootNode(*this);
}
```

src/ast/data/AstVisitor/AstVisitor.hpp

```hpp
#pragma once

class ConstantNode;
class StatementsNode;
class ProgramNode;
class RootNode;

class AstVisitor {
public:
    virtual ~AstVisitor() = default;

    virtual void visitConstantNode(const ConstantNode& node) = 0;
    virtual void visitStatementsNode(const StatementsNode& node) = 0;
    virtual void visitProgramNode(const ProgramNode& node) = 0;
    virtual void visitRootNode(const RootNode& node) = 0;
};
```

src/codegen/data/AssemblyFormatter/AssemblyFormatter.hpp

```hpp
#pragma once

#include <string>

struct AssemblyFormatterConfig {
    unsigned int labelWidth = 0;
    unsigned int instructionWidth = 0;
    unsigned int operandsWidth = 0;
};

class AssemblyFormatter {
private:
    AssemblyFormatterConfig _config;

    static std::string formatLabel(const std::string& label);
    static std::string formatComment(const std::string& comment);

    static std::string indent(unsigned int width);
    static std::string pad(const std::string& text, unsigned int width);

public:
    explicit AssemblyFormatter(AssemblyFormatterConfig config);

    [[nodiscard]] std::string sectionLine(const std::string& name) const;
    [[nodiscard]] std::string instructionLine(
        const std::string& instruction
    ) const;
    [[nodiscard]] std::string instructionLine(
        const std::string& instruction,
        const std::string& operands
    ) const;
    [[nodiscard]] std::string constantLine(
        const std::string& label,
        const std::string& size,
        const std::string& value
    ) const;

    [[nodiscard]] std::string commentLine(const std::string& comment) const;
    [[nodiscard]] std::string labelLine(const std::string& label) const;
    [[nodiscard]] std::string blankLine() const;
};
```

src/codegen/data/AssemblyFormatter/AssemblyFormatter.cpp

```cpp
#include "AssemblyFormatter.hpp"

#include <format>

AssemblyFormatter::AssemblyFormatter(AssemblyFormatterConfig config) :
    _config(config) {}

std::string AssemblyFormatter::commentLine(const std::string& comment) const {
    return formatComment(comment) + "\n";
}

std::string AssemblyFormatter::blankLine() const {
    return "\n";
}

std::string AssemblyFormatter::sectionLine(const std::string& name) const {
    return indent(_config.labelWidth) + std::format("section {}:\n", name);
}

std::string AssemblyFormatter::labelLine(const std::string& label) const {
    return formatLabel(label) + "\n";
}

std::string AssemblyFormatter::instructionLine(
    const std::string& instruction
) const {
    return indent(_config.labelWidth) + instruction + "\n";
}

std::string AssemblyFormatter::instructionLine(
    const std::string& instruction,
    const std::string& operands
) const {
    auto instructionPad = _config.instructionWidth > 0 ? _config.instructionWidth - 1 : 0;

    std::string line = indent(_config.labelWidth);
    line += pad(instruction, instructionPad);
    line += " ";
    line += operands;
    line += "\n";

    return line;
}

std::string AssemblyFormatter::constantLine(
    const std::string& label,
    const std::string& size,
    const std::string& value
) const {
    auto labelPad = _config.labelWidth > 0 ? _config.labelWidth - 1 : 0;
    auto instructionPad = _config.instructionWidth > 0 ? _config.instructionWidth - 1 : 0;

    std::string line = pad(formatLabel(label), labelPad);
    line += " ";
    line += pad(size, instructionPad);
    line += " ";
    line += value;
    line += "\n";

    return line;
}

```

```cpp
std::string AssemblyFormatter::formatLabel(const std::string& label) {
    return std::format("{}:", label);
}

std::string AssemblyFormatter::formatComment(const std::string& comment) {
    return std::format("; {}", comment);
}

std::string AssemblyFormatter::indent(unsigned int width) {
    return std::string(width, ' ');
}

std::string AssemblyFormatter::pad(
    const std::string& text,
    unsigned int width
) {
    if (text.length() >= width) {
        return text;
    }

    return text + std::string(width - text.length(), ' ');
}
```

src/codegen/data/CodeGenerator/CodeGenerator.hpp

```hpp
#pragma once

#include <AbstractSyntaxTree.hpp>
#include <AssemblyFormatter.hpp>
#include <AstNode.hpp>
#include <AstVisitor.hpp>
#include <sstream>
#include <string>

class CodeGenerator : public AstVisitor {
private:
    const AbstractSyntaxTree& _ast;
    const AssemblyFormatter& _formatter;
    std::ostringstream _out;

    void visitRootNode(const RootNode& node) override;
    void visitProgramNode(const ProgramNode& node) override;
    void visitConstantNode(const ConstantNode& node) override;
    void visitStatementsNode(const StatementsNode& node) override;

public:
    explicit CodeGenerator(
        const AbstractSyntaxTree& ast,
        const AssemblyFormatter& formatter
    );

    void generate();
    [[nodiscard]] std::string output() const;
};
```

src/codegen/data/CodeGenerator/CodeGenerator.cpp

```cpp
#include "CodeGenerator.hpp"

#include <complex>
#include <format>
#include <type_traits>
#include <variant>

CodeGenerator::CodeGenerator(
    const AbstractSyntaxTree& ast,
    const AssemblyFormatter& formatter
) :
    _ast(ast), _formatter(formatter) {}

void CodeGenerator::generate() {
    _ast.accept(*this);
}

void CodeGenerator::visitRootNode(const RootNode& node) {
    if (node.program()) {
        node.program()->accept(*this);
    }
}

void CodeGenerator::visitProgramNode(const ProgramNode& node) {
    if (!node.constants().empty()) {
        _out << _formatter.sectionLine(".data");

        for (const auto& constant : node.constants()) {
            constant->accept(*this);
        }

        _out << _formatter.blankLine();
    }

    _out << _formatter.sectionLine(".text");
    _out << _formatter.instructionLine("global", "_start");

    if (node.statements()) {
        node.statements()->accept(*this);
    }
}

void CodeGenerator::visitConstantNode(const ConstantNode& node) {
    std::visit(
        [&](const auto& value) {
            using T = std::decay_t<decltype(value)>;
            if constexpr (std::is_same_v<T, std::complex<int>>) {
                _out << _formatter.constantLine(
                    node.identifier(), "dd",
                    std::format("{}, {}", value.real(), value.imag())
                );
            } else if constexpr (std::is_same_v<T, std::complex<float>>) {
                _out << _formatter.constantLine(
                    node.identifier(), "dd",
                    std::format("{:g}, {:g}", value.real(), value.imag())
                );
            }
        },
        node.value()
    );
}

```

```
63 void CodeGenerator::visitStatementsNode(const StatementsNode&) {
64     _out << _formatter.labelLine("_start");
65     _out << _formatter.instructionLine("nop");
66     _out << _formatter.instructionLine("mov", "eax, 60");
67     _out << _formatter.instructionLine("xor", "edi, edi");
68     _out << _formatter.instructionLine("syscall");
69 }
70 
71 std::string CodeGenerator::output() const {
72     return _out.str();
73 }
```

src/declarations/data/Declaration/Declaration.hpp

```cpp
#pragma once

#include <Type.hpp>
#include <complex>
#include <cstddef>
#include <optional>
#include <string>
#include <variant>

using Value = std::variant<std::complex<int>, std::complex<float>>;

enum class DeclarationKind {
    Constant,
    Program,
};

struct Declaration {
    std::string identifier;
    DeclarationKind kind;
    std::optional<Type> type;
    std::optional<Value> value;
    size_t row = 0;
    size_t column = 0;
};
```

src/declarations/data/DeclarationsTable/DeclarationsTable.hpp

```hpp
#pragma once

#include <Declaration.hpp>
#include <optional>
#include <unordered_map>
#include <utility>
#include <vector>

class DeclarationsTable {
private:
    std::unordered_map<std::string, Declaration> _declarations;

public:
    [[nodiscard]] std::optional<Declaration> lookup(
        const std::string& identifier
    ) const;

    std::pair<const Declaration*, bool> declare(Declaration declaration);

    [[nodiscard]] std::vector<Declaration> entries() const;
};
```

src/declarations/data/DeclarationsTable/DeclarationsTable.cpp

```cpp
#include "DeclarationsTable.hpp"

#include <algorithm>
#include <ranges>

std::optional<Declaration> DeclarationsTable::lookup(
    const std::string& identifier
) const {
    if (!_declarations.contains(identifier)) {
        return std::nullopt;
    }

    return _declarations.at(identifier);
}

std::pair<const Declaration*, bool> DeclarationsTable::declare(
    Declaration declaration
) {
    auto [iterator, inserted] = _declarations.try_emplace(
        declaration.identifier, std::move(declaration)
    );

    return {&iterator->second, inserted};
}

std::vector<Declaration> DeclarationsTable::entries() const {
    auto result = std::ranges::to<std::vector<Declaration>>(
        std::views::values(_declarations)
    );

    std::ranges::sort(result, {}, &Declaration::identifier);

    return result;
}
```

src/declarations/view/DeclarationsTableView/DeclarationsTableView.hpp

```cpp
#pragma once

#include <Declaration.hpp>
#include <TableView.hpp>
#include <string>

#include "DeclarationsTable.hpp"

class DeclarationsTableView {
private:
    TableView<Declaration> _table;

public:
    DeclarationsTableView(std::string title, const DeclarationsTable& data);

    void print() const;
};
```

src/declarations/view/DeclarationsTableView/DeclarationsTableView.cpp

```cpp
#include "DeclarationsTableView.hpp"

#include <format>
#include <variant>

#include "Declaration.hpp"

namespace {
std::string extractDeclarationIdentifier(const Declaration& decl) {
    return decl.identifier;
}

std::string extractDeclarationKind(const Declaration& decl) {
    switch (decl.kind) {
        case DeclarationKind::Program:
            return "Program";
        case DeclarationKind::Constant:
            return "Constant";
        default:
            return "?";
    }
}

std::string extractDeclarationType(const Declaration& decl) {
    if (!decl.type) {
        return "";
    }

    switch (decl.type.value()) {
        case Type::ComplexInteger:
            return "Complex Integer";
        case Type::ComplexFloat:
            return "Complex Float";
        default:
            return "?";
    }
}

std::string extractDeclarationValue(const Declaration& decl) {
    if (!decl.value) {
        return "";
    }

    return std::visit(
        [](const auto& item) -> std::string {
            using T = std::decay_t<decltype(item)>;
            if constexpr (std::is_same_v<T, std::complex<int>>) {
                return std::format("{} + i*{}", item.real(), item.imag());
            } else {
                return std::format(
                    "{:.3f} + i*{:.3f}", item.real(), item.imag()
                );
            }
        },
        *decl.value
    );
}

}  // namespace

DeclarationsTableView::DeclarationsTableView(
    std::string title,
```

```
    const DeclarationsTable& data
) :
    _table(std::move(title), data.entries()) {
    _table.addColumn("Identifier", "{:<20}", extractDeclarationIdentifier)
        .addColumn("Kind", "{:<10}", extractDeclarationKind)
        .addColumn("Type", "{:<20}", extractDeclarationType)
        .addColumn("Value", "{:<30}", extractDeclarationValue);
}

void DeclarationsTableView::print() const {
    _table.print();
}
```

src/grammar/data/Rules/Rules.hpp

```hpp
#pragma once

#include <string>
#include <unordered_map>

struct Rule {
    std::string id;
    std::string input;
    std::string output;
};

enum class RuleKey {
    Axiom,
    SignalProgram,
    Program,
    Block,
    StatementsList,
    Declarations,
    ConstantDeclarations,
    ConstantDeclarationsEmpty,
    ConstantDeclarationsList,
    ConstantDeclarationsListEmpty,
    ConstantDeclaration,
    Constant,
    ComplexNumber,
    LeftPartValue,
    LeftPartEmpty,
    RightPartBase,
    RightPartExp,
    RightPartEmpty,
    ConstantIdentifier,
    ProcedureIdentifier,
    Terminal,
};

inline const std::unordered_map<RuleKey, Rule> RULES = {
    {RuleKey::Axiom, {.id = "S", .input = "", .output = "<signal-program>"}},
    {RuleKey::SignalProgram,
     {.id = "1", .input = "<signal-program>", .output = "<program>"}},
    {RuleKey::Program,
     {.id = "2",
      .input = "<program>",
      .output = "PROGRAM <procedure-identifier>; <block>."}},
    {RuleKey::Block,
     {.id = "3",
      .input = "<block>",
      .output = "<declarations> BEGIN <statements-list> END"}},
    {RuleKey::StatementsList,
     {.id = "4", .input = "<statements-list>", .output = "<empty>"}},
    {RuleKey::Declarations,
     {.id = "5",
      .input = "<declarations>",
      .output = "<constant-declarations>"}},
    {RuleKey::ConstantDeclarations,
     {.id = "6",
      .input = "<constant-declarations>",
      .output = "CONST <constant-declarations-list>"}},
    {RuleKey::ConstantDeclarationsEmpty,
     {.id = "6", .input = "<constant-declarations>", .output = "<empty>"}},
    {RuleKey::ConstantDeclarationsList,
     {.id = "7",
      .input = "<constant-declarations-list>",
```

```
63        .output = "<constant-declaration> <constant-declarations-list>"}},
64      {RuleKey::ConstantDeclarationsListEmpty,
65       {.id = "7", .input = "<constant-declarations-list>", .output = "<empty>"}},
66      {RuleKey::ConstantDeclaration,
67       {.id = "8",
68        .input = "<constant-declaration>",
69        .output = "<constant-identifier> = <constant>;"}},
70      {RuleKey::Constant,
71       {.id = "9", .input = "<constant>", .output = "'<complex-number>'"}},
72      {RuleKey::ComplexNumber,
73       {.id = "10",
74        .input = "<complex-number>",
75        .output = "<left-part> <right-part>"}},
76      {RuleKey::LeftPartValue,
77       {.id = "11", .input = "<left-part>", .output = "<unsigned-integer>"}},
78      {RuleKey::LeftPartEmpty,
79       {.id = "11", .input = "<left-part>", .output = "<empty>"}},
80      {RuleKey::RightPartBase,
81       {.id = "12", .input = "<right-part>", .output = ", <unsigned-integer>"}},
82      {RuleKey::RightPartExp,
83       {.id = "12",
84        .input = "<right-part>",
85        .output = "$EXP( <unsigned-integer> )"}},
86      {RuleKey::RightPartEmpty,
87       {.id = "12", .input = "<right-part>", .output = "<empty>"}},
88      {RuleKey::ConstantIdentifier,
89       {.id = "13", .input = "<constant-identifier>", .output = "<identifier>"}},
90      {RuleKey::ProcedureIdentifier,
91       {.id = "14", .input = "<procedure-identifier>", .output = "<identifier>"}},
92  };
```

src/parser/data/Parser/Parser.hpp

```hpp
#pragma once

#include <Logger.hpp>
#include <ParsingScope.hpp>
#include <Queue.hpp>
#include <Stack.hpp>
#include <SymbolStore.hpp>
#include <SyntaxData.hpp>
#include <SyntaxError.hpp>
#include <Tokenizer.hpp>
#include <Tree.hpp>

class Parser {
private:
    const SymbolStore& _symbols;
    Logger<SyntaxError>& _logger;

    Queue<Token> _tokens;

    Tree<SyntaxData> _tree;
    Stack<TreeNode<SyntaxData>*> _nodes;

    // 1. <signal-program> --> <program>
    void parseSignalProgram();
    // 2. <program> --> PROGRAM <procedure-identifier>; <block>.
    void parseProgram();
    // 3. <block> --> <declarations> BEGIN <statements-list> END
    void parseBlock();
    // 4. <statements-list> --> <empty>
    void parseStatementsList();
    // 5. <declarations> --> <constant-declarations>
    void parseDeclarations();
    // 6. <constant-declarations> --> CONST <constant-declarations-list> |
    // <empty>
    void parseConstantDeclarations();
    // 7. <constant-declarations-list> --> <constant-declaration>
    // <constant-declarations-list> | <empty>
    void parseConstantDeclarationsList();
    // 8. <constant-declaration> --> <constant-identifier> = <constant>;
    void parseConstantDeclaration();
    // 9. <constant> --> '<complex-number>'
    void parseConstant();
    // 10. <complex-number> --> <left-part> <right-part>
    void parseComplexNumber();
    // 11. <left-part> --> <unsigned-integer> | <empty>
    void parseLeftPart();
    // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
    // <empty>
    void parseRightPart();
    // 13. <constant-identifier> --> <identifier>
    void parseConstantIdentifier();
    // 14. <procedure-identifier> --> <identifier>
    void parseProcedureIdentifier();
    // 15. <identifier> --> <letter><string>
    void parseIdentifier(const std::string& errorMessage);
    // 17. <unsigned-integer> --> <digit><digits-string>
    void parseUnsignedInteger();

    void skipToProcedureBody();
    void skipToBlockEnd();
    void skipToNextDeclaration();

```

```
63      Token expect(const std::string& expected, const std::string& errorMessage);
64      Token expect(SymbolType expected, const std::string& errorMessage);
65
66      bool consider(const std::string& expected);
67      bool consider(SymbolType expected);
68
69      [[noreturn]] void fail(const std::string& message);
70      [[noreturn]] void fail(const std::string& message, const Token& token);
71
72      [[nodiscard]] ParsingScope grow(const SyntaxData& data);
73
74  public:
75      Parser(
76          const SymbolStore& symbols,
77          const std::vector<Token>& tokens,
78          Logger<SyntaxError>& logger
79      );
80
81      void parse();
82      const Tree<SyntaxData>& tree() const;
83  };
```

src/parser/data/Parser/Parser.cpp

```cpp
#include "Parser.hpp"

#include <Rules.hpp>

Parser::Parser(
    const SymbolStore& symbols,
    const std::vector<Token>& tokens,
    Logger<SyntaxError>& logger
) :
    _symbols(symbols),
    _logger(logger),
    _tokens(tokens),
    _tree(Tree<SyntaxData>({.rule = RuleKey::Axiom, .token = _tokens.peek()})) {
    _nodes.push(&_tree.root());
}

void Parser::parse() {
    parseSignalProgram();
}

// 1. <signal-program> --> <program>
void Parser::parseSignalProgram() {
    auto scope =
        grow({.rule = RuleKey::SignalProgram, .token = _tokens.peek()});

    try {
        parseProgram();
    } catch (const SyntaxError&) {
        // If parsing fails at the top level, we can't really recover,
        // so we just clear the remaining tokens to prevent cascading errors.
        _tokens.clear();
    }

    if (!_tokens.isEmpty()) {
        auto token = _tokens.peek();

        _logger.message({
            .message = SyntaxError::UnexpectedSymbolsAfterEndOfProgram,
            .row = token->row,
            .column = token->column,
        });
    }
}

// 2. <program> --> PROGRAM <procedure-identifier>; <block>.
void Parser::parseProgram() {
    auto scope = grow({.rule = RuleKey::Program, .token = _tokens.peek()});

    try {
        expect(Keyword::Program, SyntaxError::MustStartWithProgram);
        parseProcedureIdentifier();
        expect(Delimiter::Semicolon, SyntaxError::ExpectedSemicolon);
    } catch (const SyntaxError&) {
        skipToProcedureBody();
    }

    try {
        parseBlock();
    } catch (const SyntaxError&) {
        skipToBlockEnd();
    }

```

```
63      expect(Delimiter::Dot, SyntaxError::ExpectedDot);
64  }
65
66  // 3. <block> --> <declarations> BEGIN <statements-list> END
67  void Parser::parseBlock() {
68      auto scope = grow({.rule = RuleKey::Block, .token = _tokens.peek()});
69
70      parseDeclarations();
71      expect(Keyword::Begin, SyntaxError::ExpectedBeginKeyword);
72      parseStatementsList();
73      expect(Keyword::End, SyntaxError::ExpectedEndKeyword);
74  }
75
76  // 4. <statements-list> --> <empty>
77  void Parser::parseStatementsList() {
78      auto scope =
79          grow({.rule = RuleKey::StatementsList, .token = _tokens.peek()});
80  }
81
82  // 5. <declarations> --> <constant-declarations>
83  void Parser::parseDeclarations() {
84      auto scope = grow({.rule = RuleKey::Declarations, .token = _tokens.peek()});
85
86      parseConstantDeclarations();
87  }
88
89  // 6. <constant-declarations> --> CONST <constant-declarations-list> | <empty>
90  void Parser::parseConstantDeclarations() {
91      if (consider(Keyword::Const)) {
92          auto scope = grow(
93              {.rule = RuleKey::ConstantDeclarations, .token = _tokens.peek()}
94          );
95
96          expect(Keyword::Const, SyntaxError::ExpectedConstKeyword);
97          parseConstantDeclarationsList();
98      } else {
99          auto scope = grow(
100             {.rule = RuleKey::ConstantDeclarationsEmpty,
101              .token = _tokens.peek()}
102         );
103     }
104 }
105
106 // 7. <constant-declarations-list> --> <constant-declaration>
107 // <constant-declarations-list> | <empty>
108 void Parser::parseConstantDeclarationsList() {
109     if (!consider(Keyword::Begin)) {
110         auto scope = grow(
111             {.rule = RuleKey::ConstantDeclarationsList, .token = _tokens.peek()}
112         );
113
114         try {
115             parseConstantDeclaration();
116         } catch (const SyntaxError&) {
117             skipToNextDeclaration();
118         }
119
120         parseConstantDeclarationsList();
121     } else {
122         auto scope = grow(
123             {.rule = RuleKey::ConstantDeclarationsListEmpty,
124              .token = _tokens.peek()}
125         );
126     }
127 }
128
129 // 8. <constant-declaration> --> <constant-identifier> = <constant>;
```

```
130 void Parser::parseConstantDeclaration() {
131     auto scope =
132         grow({.rule = RuleKey::ConstantDeclaration, .token = _tokens.peek()});
133
134     parseConstantIdentifier();
135     expect(Delimiter::Equals, SyntaxError::ExpectedEquals);
136     parseConstant();
137     try {
138         expect(Delimiter::Semicolon, SyntaxError::ExpectedConstantSemicolon);
139     } catch (const SyntaxError&) {
140         // Implicitly add a semicolon to recover and continue parsing the next
141         // declarations.
142         // TODO: check that this does not backfire for severely mangled
143         // declarations.
144     }
145 }
146
147 // 9. <constant> --> '<complex-number>'
148 void Parser::parseConstant() {
149     auto scope = grow({.rule = RuleKey::Constant, .token = _tokens.peek()});
150
151     expect(Delimiter::Quote, SyntaxError::ExpectedOpeningQuote);
152     parseComplexNumber();
153     expect(Delimiter::Quote, SyntaxError::ExpectedClosingQuote);
154 }
155
156 // 10. <complex-number> --> <left-part> <right-part>
157 void Parser::parseComplexNumber() {
158     auto scope =
159         grow({.rule = RuleKey::ComplexNumber, .token = _tokens.peek()});
160
161     parseLeftPart();
162     parseRightPart();
163 }
164
165 // 11. <left-part> --> <unsigned-integer> | <empty>
166 void Parser::parseLeftPart() {
167     if (consider(SymbolType::Literal)) {
168         auto scope =
169             grow({.rule = RuleKey::LeftPartValue, .token = _tokens.peek()});
170
171         parseUnsignedInteger();
172     } else {
173         auto scope =
174             grow({.rule = RuleKey::LeftPartEmpty, .token = _tokens.peek()});
175     }
176 }
177
178 // 12. <right-part> --> ,<unsigned-integer> | $EXP( <unsigned-integer> ) |
179 // <empty>
180 void Parser::parseRightPart() {
181     if (consider(Delimiter::Comma)) {
182         auto scope =
183             grow({.rule = RuleKey::RightPartBase, .token = _tokens.peek()});
184
185         expect(Delimiter::Comma, SyntaxError::ExpectedComma);
186         parseUnsignedInteger();
187         return;
188     }
189
190     if (consider(MultiDelimiter::Exp)) {
191         auto scope =
192             grow({.rule = RuleKey::RightPartExp, .token = _tokens.peek()});
193
194         expect(MultiDelimiter::Exp, SyntaxError::ExpectedExp);
195         expect(Delimiter::OpenParenthesis, SyntaxError::ExpectedOpenParen);
196         parseUnsignedInteger();
```

```
197         expect(Delimiter::CloseParenthesis, SyntaxError::ExpectedCloseParen);
198         return;
199     }
200
201     auto scope =
202         grow({.rule = RuleKey::RightPartEmpty, .token = _tokens.peek()});
203 }
204
205 // 13. <constant-identifier> --> <identifier>
206 void Parser::parseConstantIdentifier() {
207     auto scope =
208         grow({.rule = RuleKey::ConstantIdentifier, .token = _tokens.peek()});
209
210     parseIdentifier(SyntaxError::ExpectedConstantIdentifier);
211 }
212
213 // 14. <procedure-identifier> --> <identifier>
214 void Parser::parseProcedureIdentifier() {
215     auto scope =
216         grow({.rule = RuleKey::ProcedureIdentifier, .token = _tokens.peek()});
217
218     parseIdentifier(SyntaxError::ExpectedProcedureIdentifier);
219 }
220
221 // 15. <identifier> --> <letter><string>  (terminal)
222 void Parser::parseIdentifier(const std::string& errorMessage) {
223     auto token = expect(SymbolType::Identifier, errorMessage);
224
225     auto scope = grow({.rule = RuleKey::Terminal, .token = token});
226 }
227
228 // 17. <unsigned-integer> --> <digit><digits-string>  (terminal)
229 void Parser::parseUnsignedInteger() {
230     auto token =
231         expect(SymbolType::Literal, SyntaxError::ExpectedUnsignedInteger);
232
233     auto scope = grow({.rule = RuleKey::Terminal, .token = token});
234 }
235
236 // --- Error recovery ---
237
238 void Parser::skipToProcedureBody() {
239     auto found = _tokens.popUntil([&](const Token& token) {
240         const auto& symbol = _symbols.lookup(token.code);
241         return symbol == Keyword::Const || symbol == Keyword::Begin;
242     });
243
244     if (!found) {
245         fail(SyntaxError::UnexpectedEndOfFile);
246     }
247 }
248
249 void Parser::skipToBlockEnd() {
250     auto found = _tokens.popUntil([&](const Token& token) {
251         const auto& symbol = _symbols.lookup(token.code);
252         return symbol == Keyword::End || symbol == Delimiter::Dot;
253     });
254
255     if (!found) {
256         fail(SyntaxError::UnexpectedEndOfFile);
257     }
258
259     if (_symbols.lookup(found->code) == Keyword::End) {
260         _tokens.pop();
261     }
262 }
263
```

```
264 void Parser::skipToNextDeclaration() {
265     auto found = _tokens.popUntil([&](const Token& token) {
266         const auto& symbol = _symbols.lookup(token.code);
267         return symbol == Delimiter::Semicolon || symbol == Keyword::Begin;
268     });
269 
270     if (!found) {
271         fail(SyntaxError::UnexpectedEndOfFile);
272     }
273 
274     if (_symbols.lookup(found->code) == Delimiter::Semicolon) {
275         _tokens.pop();
276     }
277 }
278 
279 // --- Utilities ---
280 
281 Token Parser::expect(
282     const std::string& expected,
283     const std::string& errorMessage
284 ) {
285     auto token = _tokens.peek();
286 
287     if (!token) {
288         fail(errorMessage);
289     }
290 
291     if (_symbols.lookup(token->code) != expected) {
292         fail(errorMessage, *token);
293     }
294 
295     return *_tokens.pop();
296 }
297 
298 Token Parser::expect(SymbolType expected, const std::string& errorMessage) {
299     auto token = _tokens.peek();
300 
301     if (!token) {
302         fail(errorMessage);
303     }
304 
305     if (_symbols.lookupType(token->code) != expected) {
306         fail(errorMessage, *token);
307     }
308 
309     return *_tokens.pop();
310 }
311 
312 bool Parser::consider(const std::string& expected) {
313     auto token = _tokens.peek();
314 
315     return token && _symbols.lookup(token->code) == expected;
316 }
317 
318 bool Parser::consider(SymbolType expected) {
319     auto token = _tokens.peek();
320 
321     return token && _symbols.lookupType(token->code) == expected;
322 }
323 
324 void Parser::fail(const std::string& message) {
325     SyntaxError error{.message = message, .row = 0, .column = 0};
326 
327     _logger.message(error);
328 
329     throw error;
330 }
```

```
331
332 void Parser::fail(const std::string& message, const Token& token) {
333     SyntaxError error{
334         .message = message, .row = token.row, .column = token.column
335     };
336
337     _logger.message(error);
338
339     throw error;
340 }
341
342 ParsingScope Parser::grow(const SyntaxData& data) {
343     auto parentNode = _nodes.peek();
344     auto& node = (*parentNode)->grow(data);
345     _nodes.push(&node);
346     return ParsingScope(_nodes);
347 }
348
349 const Tree<SyntaxData>& Parser::tree() const {
350     return _tree;
351 }
```

src/parser/data/ParsingScope/ParsingScope.hpp

```hpp
#pragma once

#include <Stack.hpp>
#include <SyntaxData.hpp>
#include <Tree.hpp>

class ParsingScope {
    Stack<TreeNode<SyntaxData>*>* _stack;

public:
    explicit ParsingScope(Stack<TreeNode<SyntaxData>*>& stack);
    ~ParsingScope();

    ParsingScope(ParsingScope&& other) noexcept;
    ParsingScope(const ParsingScope&) = delete;
    ParsingScope& operator=(const ParsingScope&) = delete;
    ParsingScope& operator=(ParsingScope&&) = delete;
};
```

src/parser/data/ParsingScope/ParsingScope.cpp

```cpp
#include "ParsingScope.hpp"

ParsingScope::ParsingScope(Stack<TreeNode<SyntaxData>*>& stack) :
    _stack(&stack) {}

ParsingScope::~ParsingScope() {
    if (_stack) {
        _stack->pop();
    }
}

ParsingScope::ParsingScope(ParsingScope&& other) noexcept :
    _stack(other._stack) {
    other._stack = nullptr;
}
```

src/parser/data/SyntaxData/SyntaxData.hpp

```cpp
#pragma once

#include <optional>

#include <Rules.hpp>
#include <Tokenizer.hpp>

struct SyntaxData {
    RuleKey rule;
    std::optional<Token> token;
};
```

src/parser/data/SyntaxError/SyntaxError.hpp

```hpp
#pragma once

#include <cstddef>
#include <string>

struct SyntaxError {
    std::string message;
    size_t row;
    size_t column;

    // Error message constants
    static constexpr auto MustStartWithProgram =
        "Program must start with 'PROGRAM' keyword";
    static constexpr auto ExpectedProcedureIdentifier =
        "Expected procedure identifier after 'PROGRAM' keyword";
    static constexpr auto ExpectedSemicolon =
        "Expected ';' after procedure identifier";
    static constexpr auto ExpectedDot =
        "Expected '.' at the end of the program";
    static constexpr auto ExpectedBeginKeyword =
        "Expected 'BEGIN' keyword before statements list";
    static constexpr auto ExpectedEndKeyword =
        "Expected 'END' keyword at the end of the block";
    static constexpr auto ExpectedConstKeyword =
        "Expected 'CONST' keyword before constant declarations";
    static constexpr auto ExpectedIdentifier = "Expected identifier";
    static constexpr auto ExpectedConstantIdentifier =
        "Expected constant identifier in constant declaration";
    static constexpr auto ExpectedEquals =
        "Expected '=' in constant declaration";
    static constexpr auto ExpectedConstantSemicolon =
        "Expected ';' at the end of constant declaration";
    static constexpr auto ExpectedOpeningQuote =
        "Expected ''' (single quote) at the beginning of constant declaration";
    static constexpr auto ExpectedClosingQuote =
        "Expected ''' (single quote) at the end of constant declaration";
    static constexpr auto ExpectedUnsignedInteger =
        "Expected unsigned integer literal";
    static constexpr auto ExpectedComma = "Expected ',' in complex number";
    static constexpr auto ExpectedExp = "Expected '$EXP' in complex number";
    static constexpr auto ExpectedOpenParen =
        "Expected '(' after '$EXP' in complex number";
    static constexpr auto ExpectedCloseParen =
        "Expected ')' after exponent in complex number";
    static constexpr auto UnexpectedEndOfFile = "Unexpected end of file";
    static constexpr auto UnexpectedSymbolsAfterEndOfProgram =
        "Unexpected symbols after end of program";
};
```

src/parser/view/DotTreeView/DotTreeView.hpp

```hpp
#pragma once

#include <string>

#include <Parser.hpp>
#include <SymbolStore.hpp>

class DotTreeView {
private:
    const Tree<SyntaxData>& _tree;
    const SymbolStore& _symbols;

    std::string nodeLabel(const SyntaxData& data) const;
    std::string edgeLabel(const SyntaxData& data) const;

    void writeNode(
        std::ostream& out,
        const TreeNode<SyntaxData>& node,
        int& id
    ) const;

public:
    DotTreeView(const Tree<SyntaxData>& tree, const SymbolStore& symbols);

    void write(const std::string& filename) const;
};
```

src/parser/view/DotTreeView/DotTreeView.cpp

```cpp
#include "DotTreeView.hpp"

#include <fstream>

#include <Rules.hpp>

DotTreeView::DotTreeView(
    const Tree<SyntaxData>& tree,
    const SymbolStore& symbols
) :
    _tree(tree), _symbols(symbols) {}

std::string DotTreeView::nodeLabel(const SyntaxData& data) const {
    if (data.rule == RuleKey::Terminal) {
        return _symbols.lookup(data.token->code);
    }
    return RULES.at(data.rule).output;
}

std::string DotTreeView::edgeLabel(const SyntaxData& data) const {
    if (data.rule == RuleKey::Terminal) {
        return "T";
    }
    return RULES.at(data.rule).id;
}

void DotTreeView::writeNode(
    std::ostream& out,
    const TreeNode<SyntaxData>& node,
    int& id
) const {
    int nodeId = id++;
    const auto& data = node.data();

    out << "    n" << nodeId << " [label=\"" << nodeLabel(data) << "\"]\n";

    for (const auto& child : node.children()) {
        int childId = id;
        writeNode(out, *child, id);

        int ruleId = id++;
        out << "    r" << ruleId << " [label=\"" << edgeLabel(child->data())
            << "\" shape=circle width=0.3 fixedsize=true"
            << " fontsize=10]\n";

        out << "    n" << nodeId << " -> r" << ruleId
            << " [arrowhead=none]\n";
        out << "    r" << ruleId << " -> n" << childId << "\n";
    }
}

void DotTreeView::write(const std::string& filename) const {
    std::ofstream out(filename);

    out << "digraph SyntaxTree {\n"
        << "    rankdir=TB\n"
        << "    node [shape=none fontname=\"Consolas\" fontsize=11]\n"
        << "    edge [arrowsize=0.6]\n\n";

    int id = 0;
    writeNode(out, _tree.root(), id);

```

```
63      out << "}\n";
64  }
```

src/parser/view/SyntaxTreeView/SyntaxTreeView.hpp

```hpp
#pragma once

#include <string>

#include <Parser.hpp>
#include <SymbolStore.hpp>
#include <TreeView.hpp>

class SyntaxTreeView {
private:
    TreeView<SyntaxData> _tree;

public:
    SyntaxTreeView(
        std::string title,
        const Tree<SyntaxData>& tree,
        const SymbolStore& symbols
    );

    void print() const;
};
```

src/parser/view/SyntaxTreeView/SyntaxTreeView.cpp

```cpp
#include "SyntaxTreeView.hpp"

#include <format>

#include <Rules.hpp>

SyntaxTreeView::SyntaxTreeView(
    std::string title,
    const Tree<SyntaxData>& tree,
    const SymbolStore& symbols
) :
    _tree(std::move(title), tree) {
    _tree
        .setNodeFormatter([&symbols](const auto& data) {
            if (data.rule == RuleKey::Terminal) {
                return symbols.lookup(data.token->code);
            }
            return RULES.at(data.rule).output;
        })
        .setEdgeFormatter([](const auto& data) {
            if (data.rule == RuleKey::Terminal) {
                return std::string("(T)");
            }
            return std::format("({})", RULES.at(data.rule).id);
        });
}

void SyntaxTreeView::print() const {
    _tree.print();
}
```

src/semantics/data/SemanticAnalyzer/SemanticAnalyzer.hpp

```hpp
#pragma once

#include <AbstractSyntaxTree.hpp>
#include <AstVisitor.hpp>
#include <Declaration.hpp>
#include <DeclarationsTable.hpp>
#include <Logger.hpp>
#include <SemanticError.hpp>

class SemanticAnalyzer : public AstVisitor {
private:
    const AbstractSyntaxTree& _ast;
    Logger<SemanticError>& _logger;

    DeclarationsTable _declarations;

public:
    SemanticAnalyzer(
        const AbstractSyntaxTree& ast, Logger<SemanticError>& logger
    );

    void analyze();
    [[nodiscard]] const DeclarationsTable& declarations() const;

    void visitRootNode(const RootNode& node) override;
    void visitProgramNode(const ProgramNode& node) override;
    void visitConstantNode(const ConstantNode& node) override;
    void visitStatementsNode(const StatementsNode& node) override;
};
```

src/semantics/data/SemanticAnalyzer/SemanticAnalyzer.cpp

```cpp
#include "SemanticAnalyzer.hpp"

#include <AstNode.hpp>
#include <Declaration.hpp>
#include <SemanticError.hpp>
#include <complex>
#include <variant>

namespace {

Type typeOf(const Value& value) {
    return std::visit(
        [](const auto& alternative) {
            using T = std::decay_t<decltype(alternative)>;
            if constexpr (std::is_same_v<T, std::complex<float>>) {
                return Type::ComplexFloat;
            } else {
                return Type::ComplexInteger;
            }
        },
        value
    );
}

}  // namespace

SemanticAnalyzer::SemanticAnalyzer(
    const AbstractSyntaxTree& ast,
    Logger<SemanticError>& logger
) :
    _ast(ast), _logger(logger) {}

void SemanticAnalyzer::analyze() {
    _ast.accept(*this);
}

const DeclarationsTable& SemanticAnalyzer::declarations() const {
    return _declarations;
}

void SemanticAnalyzer::visitRootNode(const RootNode& node) {
    if (node.program()) {
        node.program()->accept(*this);
    }
}

void SemanticAnalyzer::visitProgramNode(const ProgramNode& node) {
    auto [existing, inserted] = _declarations.declare({
        .identifier = node.identifier(),
        .kind = DeclarationKind::Program,
        .type = std::nullopt,
        .value = std::nullopt,
        .row = node.row(),
        .column = node.column(),
    });

    if (!inserted) {
        _logger.message({
            .message = SemanticError::DuplicateIdentifier(
                node.identifier(), existing->row, existing->column
            ),
            .row = node.row(),
```

```
            .column = node.column(),
        });
    }

    for (const auto& constant : node.constants()) {
        constant->accept(*this);
    }
}

void SemanticAnalyzer::visitStatementsNode(const StatementsNode& node) {}

void SemanticAnalyzer::visitConstantNode(const ConstantNode& node) {
    auto [existing, inserted] = _declarations.declare({
        .identifier = node.identifier(),
        .kind = DeclarationKind::Constant,
        .type = typeOf(node.value()),
        .value = node.value(),
        .row = node.row(),
        .column = node.column(),
    });

    if (!inserted) {
        _logger.message({
            .message = SemanticError::DuplicateIdentifier(
                node.identifier(), existing->row, existing->column
            ),
            .row = node.row(),
            .column = node.column(),
        });
    }
}
```

src/semantics/data/SemanticError/SemanticError.hpp

```hpp
#pragma once

#include <cstddef>
#include <format>
#include <string>

struct SemanticError {
    std::string message;
    size_t row;
    size_t column;

    // Error message constants
    static auto DuplicateIdentifier(
        std::string identifier,
        size_t originalRow,
        size_t originalColumn
    ) {
        return std::format(
            "Identifier '{}' is already declared at [{}:{}]", identifier,
            originalRow + 1, originalColumn + 1
        );
    };
};
```

src/symbols/symbols.constants.hpp

```cpp
#pragma once

#include <cstddef>

constexpr size_t ASCII_TABLE_SIZE = 256;

namespace Keyword {
constexpr auto Program = "PROGRAM";
constexpr auto Const = "CONST";
constexpr auto Begin = "BEGIN";
constexpr auto End = "END";
}  // namespace Keyword

namespace MultiDelimiter {
constexpr auto Exp = "$EXP";
}

namespace Delimiter {
constexpr auto Semicolon = ";";
constexpr auto Dot = ".";
constexpr auto Equals = "=";
constexpr auto Quote = "'";
constexpr auto Comma = ",";
constexpr auto OpenParenthesis = "(";
constexpr auto CloseParenthesis = ")";
}  // namespace Delimiter
```

src/symbols/data/SymbolStore/SymbolStore.hpp

```cpp
#pragma once

#include <array>
#include <string>
#include <unordered_map>
#include <vector>

#include "symbols.constants.hpp"

enum class SymbolType {
    Ascii,
    MultiDelimiter,
    Keyword,
    Literal,
    Identifier,
};

const size_t DELIMITERS_OFFSET = 0;
const size_t MULTI_DELIMITERS_OFFSET = ASCII_TABLE_SIZE;
const size_t KEYWORDS_OFFSET = 300;
const size_t LITERALS_OFFSET = 500;
const size_t IDENTIFIERS_OFFSET = 1000;
const size_t MAX_TOKENS = 2000;

class SymbolStore {
private:
    std::array<std::string, MAX_TOKENS> symbols;
    std::unordered_map<std::string, size_t> _multiDelimiters;
    std::unordered_map<std::string, size_t> _keywords;
    std::unordered_map<std::string, size_t> _identifiers;
    std::unordered_map<std::string, size_t> _literals;

    void declareAscii(char character);
    void declareMultiDelimiter(const std::string& delimiter);
    void declareKeyword(const std::string& keyword);

public:
    SymbolStore();

    size_t resolveMultiDelimiter(const std::string& delimiter);
    size_t resolveKeyword(const std::string& keyword);
    size_t resolveIdentifier(const std::string& identifier);
    size_t resolveLiteral(const std::string& literal);

    bool isKeyword(const std::string& token) const;
    bool isMultiDelimiter(const std::string& token) const;

    const std::string& lookup(size_t code) const;
    SymbolType lookupType(size_t code) const;
    std::vector<std::pair<std::string, size_t>> identifiers() const;
    std::vector<std::pair<std::string, size_t>> literals() const;
};
```

src/symbols/data/SymbolStore/SymbolStore.cpp

```cpp
#include "SymbolStore.hpp"

#include <algorithm>
#include <format>
#include <stdexcept>

SymbolStore::SymbolStore() {
    // Initialize ASCII symbols
    for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
        declareAscii(static_cast<char>(i));
    }

    // Initialize multi-character delimiters
    declareMultiDelimiter(MultiDelimiter::Exp);

    // Initialize keywords
    declareKeyword(Keyword::Program);
    declareKeyword(Keyword::Const);
    declareKeyword(Keyword::Begin);
    declareKeyword(Keyword::End);
}

void SymbolStore::declareAscii(const char character) {
    symbols[static_cast<unsigned char>(character)] = std::string(1, character);
}

void SymbolStore::declareMultiDelimiter(const std::string& delimiter) {
    if (_multiDelimiters.contains(delimiter)) {
        throw std::invalid_argument(
            std::format("Multi-delimiter \'{}\' is already declared", delimiter)
        );
    }

    size_t code = MULTI_DELIMITERS_OFFSET + _multiDelimiters.size();

    if (code >= KEYWORDS_OFFSET) {
        throw std::overflow_error(
            "Exceeded maximum number of multi-delimiters"
        );
    }

    _multiDelimiters[delimiter] = code;
    symbols[code] = delimiter;
}

size_t SymbolStore::resolveMultiDelimiter(const std::string& delimiter) {
    if (!_multiDelimiters.contains(delimiter)) {
        throw std::invalid_argument(
            std::format("\'{}\' is not a multi-delimiter", delimiter)
        );
    }

    return _multiDelimiters[delimiter];
}

void SymbolStore::declareKeyword(const std::string& keyword) {
    if (_keywords.contains(keyword)) {
        throw std::invalid_argument(
            std::format("Keyword \'{}\' is already declared", keyword)
        );
    }

```

```
    size_t code = KEYWORDS_OFFSET + _keywords.size();

    if (code >= LITERALS_OFFSET) {
        throw std::overflow_error("Exceeded maximum number of keywords");
    }

    _keywords[keyword] = code;
    symbols[code] = keyword;
}

size_t SymbolStore::resolveKeyword(const std::string& keyword) {
    if (!_keywords.contains(keyword)) {
        throw std::invalid_argument(
            std::format("\'{}\' is not a keyword", keyword)
        );
    }

    return _keywords[keyword];
}

size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
    if (!_identifiers.contains(identifier)) {
        size_t code = IDENTIFIERS_OFFSET + _identifiers.size();

        if (code >= MAX_TOKENS) {
            throw std::overflow_error("Exceeded maximum number of identifiers");
        }

        _identifiers[identifier] = code;
        symbols[code] = identifier;
    }

    return _identifiers[identifier];
}

size_t SymbolStore::resolveLiteral(const std::string& literal) {
    if (!_literals.contains(literal)) {
        size_t code = LITERALS_OFFSET + _literals.size();

        if (code >= IDENTIFIERS_OFFSET) {
            throw std::overflow_error("Exceeded maximum number of literals");
        }

        _literals[literal] = code;
        symbols[code] = literal;
    }

    return _literals[literal];
}

bool SymbolStore::isKeyword(const std::string& token) const {
    return _keywords.contains(token);
}

bool SymbolStore::isMultiDelimiter(const std::string& token) const {
    return _multiDelimiters.contains(token);
}

SymbolType SymbolStore::lookupType(size_t code) const {
    if (code < MULTI_DELIMITERS_OFFSET) {
        return SymbolType::Ascii;
    }

    if (code < KEYWORDS_OFFSET) {
        return SymbolType::MultiDelimiter;
    }

```

```
    if (code < LITERALS_OFFSET) {
        return SymbolType::Keyword;
    }

    if (code < IDENTIFIERS_OFFSET) {
        return SymbolType::Literal;
    }

    if (code < MAX_TOKENS) {
        return SymbolType::Identifier;
    }

    throw std::out_of_range(
        std::format("Symbol code {} is out of range", code)
    );
}

const std::string& SymbolStore::lookup(size_t code) const {
    if (code >= symbols.size()) {
        throw std::out_of_range(
            std::format("Symbol code {} is out of range", code)
        );
    }

    return symbols[code];
}

std::vector<std::pair<std::string, size_t>> SymbolStore::identifiers() const {
    std::vector<std::pair<std::string, size_t>> result;

    for (const auto& pair : _identifiers) {
        result.emplace_back(pair.first, pair.second);
    }

    std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
        return a.second < b.second;
    });

    return result;
}

std::vector<std::pair<std::string, size_t>> SymbolStore::literals() const {
    std::vector<std::pair<std::string, size_t>> result;

    for (const auto& pair : _literals) {
        result.emplace_back(pair.first, pair.second);
    }

    std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) {
        return a.second < b.second;
    });

    return result;
}
```

src/tokenizer/data/CharacterAttributes/CharacterAttributes.hpp

```cpp
#pragma once

#include <array>

#include "symbols.constants.hpp"

enum class Attribute {
    Whitespace,
    Digit,
    Letter,
    Delimiter,
    MultiDelimiter,
    Comment,
    Invalid,
};

class CharacterAttributes {
private:
    std::array<Attribute, ASCII_TABLE_SIZE> attributes;

public:
    CharacterAttributes();

    [[nodiscard]] Attribute lookup(char character) const;
};
```

src/tokenizer/data/CharacterAttributes/CharacterAttributes.cpp

```cpp
#include "CharacterAttributes.hpp"

CharacterAttributes::CharacterAttributes() {
    for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
        attributes[i] = Attribute::Invalid;
    }

    // Whitespace
    attributes[' '] = Attribute::Whitespace;
    attributes['\t'] = Attribute::Whitespace;
    attributes['\n'] = Attribute::Whitespace;
    attributes['\v'] = Attribute::Whitespace;
    attributes['\f'] = Attribute::Whitespace;
    attributes['\r'] = Attribute::Whitespace;

    // Digits
    for (char ch = '0'; ch <= '9'; ch++) {
        attributes[ch] = Attribute::Digit;
    }

    // Letters (A-Z)
    for (char ch = 'A'; ch <= 'Z'; ch++) {
        attributes[ch] = Attribute::Letter;
    }

    // Delimiters
    attributes[';'] = Attribute::Delimiter;
    attributes['.'] = Attribute::Delimiter;
    attributes['='] = Attribute::Delimiter;
    attributes['\''] = Attribute::Delimiter;
    attributes[','] = Attribute::Delimiter;
    attributes[')'] = Attribute::Delimiter;

    // Multi-delimiters
    attributes['$'] = Attribute::MultiDelimiter;

    // Comment start
    attributes['('] = Attribute::Comment;
}

Attribute CharacterAttributes::lookup(char character) const {
    return attributes[static_cast<unsigned char>(character)];
}
```

src/tokenizer/data/Cursor/Cursor.hpp

```hpp
#pragma once

#include <cstdint>

class Cursor {
private:
    uint8_t tabSize = 4;

    unsigned int _row = 0;
    unsigned int _column = 0;

public:
    Cursor();
    Cursor(uint8_t tabSize);

    void advance(char character);
    [[nodiscard]] unsigned int row() const;
    [[nodiscard]] unsigned int column() const;
};
```

src/tokenizer/data/Cursor/Cursor.cpp

```cpp
#include "Cursor.hpp"

Cursor::Cursor() : Cursor(4) {}

Cursor::Cursor(uint8_t tabSize) : tabSize(tabSize) {};

void Cursor::advance(char character) {
    switch (character) {
        case '\r':
            break;

        case '\n':
            _row++;
            _column = 0;
            break;

        case '\t':
            _column += tabSize - _column % tabSize;
            break;

        default:
            _column++;
            break;
    }
}

unsigned int Cursor::row() const {
    return _row;
}

unsigned int Cursor::column() const {
    return _column;
}
```

src/tokenizer/data/FileSource/FileSource.hpp

```cpp
#pragma once

#include <string>
#include <fstream>
#include "Source.hpp"

class FileSource : public Source {
private:
    std::ifstream file;
    char character;

public:
    FileSource(const std::string& filePath);

    char current() override;
    char read() override;
    bool done() override;
};
```

src/tokenizer/data/FileSource/FileSource.cpp

```cpp
#include "FileSource.hpp"

#include <stdexcept>

FileSource::FileSource(const std::string& filePath) :
    file(filePath),
    character(static_cast<char>(file.get())) {
    if (!file.is_open()) {
        throw std::runtime_error("Could not open file: " + filePath);
    }
}

char FileSource::current() {
    return character;
}

char FileSource::read() {
    _cursor.advance(character);
    character = static_cast<char>(file.get());

    return character;
}

bool FileSource::done() {
    return file.eof();
}
```

src/tokenizer/data/LexicalError/LexicalError.hpp

```hpp
#pragma once

#include <cstddef>
#include <format>
#include <string>

struct LexicalError {
    std::string message;
    size_t row;
    size_t column;

    static auto InvalidMultiDelimiter(const std::string& token) {
        return std::format("'{}' is not a valid multi-delimiter", token);
    }

    static auto UnclosedComment() {
        return std::string("Comment not closed");
    }

    static auto InvalidCharacter(char character) {
        return std::format("Invalid character '{}'", character);
    }
};
```

src/tokenizer/data/Source/Source.hpp

```hpp
#pragma once

#include <Cursor.hpp>

class Source {
protected:
    Cursor _cursor;

public:
    Source();
    Source(Cursor cursor);

    virtual ~Source() = default;

    virtual char current() = 0;
    virtual char read() = 0;
    virtual bool done() = 0;

    [[nodiscard]] const Cursor& cursor() const;
};
```

src/tokenizer/data/Source/Source.cpp

```cpp
#include "Source.hpp"

Source::Source() = default;

Source::Source(Cursor cursor) : _cursor(cursor) {}

const Cursor& Source::cursor() const {
    return _cursor;
}
```

src/tokenizer/data/StringSource/StringSource.hpp

```hpp
#pragma once

#include <string>
#include "Source.hpp"

class StringSource : public Source {
private:
    std::string source;
    std::string::iterator iterator;

public:
    StringSource(std::string source);

    char current() override;
    char read() override;
    bool done() override;
};
```

src/tokenizer/data/StringSource/StringSource.cpp

```cpp
#include "StringSource.hpp"

StringSource::StringSource(std::string source) :
    source(std::move(source)),
    iterator(this->source.begin()) {}

char StringSource::current() {
    return *iterator;
}

char StringSource::read() {
    _cursor.advance(*iterator);
    iterator++;

    return *iterator;
}

bool StringSource::done() {
    return iterator == source.end();
}
```

src/tokenizer/data/Token/Token.hpp

```hpp
#pragma once

#include <cstddef>

struct Token {
    size_t code;
    size_t row;
    size_t column;
};
```

src/tokenizer/data/Tokenizer/Tokenizer.hpp

```hpp
#pragma once

#include <vector>

#include "CharacterAttributes.hpp"
#include "LexicalError.hpp"
#include "Logger.hpp"
#include "Source.hpp"
#include "SymbolStore.hpp"
#include "Token.hpp"

class Tokenizer {
private:
    Source& _source;
    SymbolStore& _symbols;
    CharacterAttributes& _attributes;
    Logger<LexicalError>& _logger;

    char _character;
    std::string _token;
    size_t _code;

    std::vector<Token> _tokens;

    void scanWhitespaces();
    void scanInteger();
    void scanString();
    void scanComment();
    void scanMultiDelimiter();
    void scanDelimiter();
    void scanInvalid();

public:
    Tokenizer(
        Source& source,
        SymbolStore& symbols,
        CharacterAttributes& attributes,
        Logger<LexicalError>& logger
    );

    void scan();

    void addToken(const Token& token);
    void addError(const LexicalError& error);

    [[nodiscard]] const std::vector<Token>& tokens() const;
};
```

src/tokenizer/data/Tokenizer/Tokenizer.cpp

```cpp
#include "Tokenizer.hpp"

#include "CharacterAttributes.hpp"

// Automata state: START
Tokenizer::Tokenizer(
    Source& source,
    SymbolStore& symbols,
    CharacterAttributes& attributes,
    Logger<LexicalError>& logger
) :
    _source(source),
    _symbols(symbols),
    _attributes(attributes),
    _logger(logger),
    _character(source.current()) {}

void Tokenizer::scan() {
    // Automata state: LOOP
    while (!_source.done()) {
        switch (_attributes.lookup(_character)) {
            // Automata state: WHITESPACE
            case Attribute::Whitespace:
                scanWhitespaces();
                break;

            // Automata state: INTEGER
            case Attribute::Digit:
                scanInteger();
                break;

            // Automata state: STRING
            case Attribute::Letter:
                scanString();
                break;

            // Automata state: MULTI_DELIMITER
            case Attribute::MultiDelimiter:
                scanMultiDelimiter();
                break;

            // Automata state: COMMENT_START
            case Attribute::Comment:
                scanComment();
                break;

            // Automata state: DELIMITER
            case Attribute::Delimiter:
                scanDelimiter();
                break;

            // Automata state: ERROR
            case Attribute::Invalid:
                scanInvalid();
                break;
        }
    }
    // Automata state: END
}

void Tokenizer::scanWhitespaces() {
    while (!_source.done() &&
```

```
           _attributes.lookup(_character) == Attribute::Whitespace) {
        _character = _source.read();
    }
}

void Tokenizer::scanInteger() {
    while (!_source.done() && _attributes.lookup(_character) == Attribute::Digit) {
        _token += _character;
        _character = _source.read();
    };

    // Automata state: WRITE
    _code = _symbols.resolveLiteral(_token);

    addToken({
        .code = _code,
        .row = _source.cursor().row(),
        .column = _source.cursor().column() -
                  _token.length(),  // Point to first character of the token
    });

    _token.clear();
}

void Tokenizer::scanString() {
    while (!_source.done() &&
           (_attributes.lookup(_character) == Attribute::Letter ||
            _attributes.lookup(_character) == Attribute::Digit)) {
        _token += _character;
        _character = _source.read();
    }

    // Automata state: WRITE
    if (_symbols.isKeyword(_token)) {
        _code = _symbols.resolveKeyword(_token);
    } else {
        _code = _symbols.resolveIdentifier(_token);
    }

    addToken({
        .code = _code,
        .row = _source.cursor().row(),
        .column = _source.cursor().column() -
                  _token.length(),  // Point to first character of the token
    });

    _token.clear();
}

void Tokenizer::scanMultiDelimiter() {
    size_t startRow = _source.cursor().row();
    size_t startCol = _source.cursor().column();

    _token += _character;
    _character = _source.read();

    if (_source.done() || _attributes.lookup(_character) != Attribute::Letter) {
        addError({
            .message = LexicalError::InvalidMultiDelimiter(_token),
            .row = startRow,
            .column = startCol,
        });

        _token.clear();

        return;
    }
```

```
130
131     while (!_source.done() &&
132            (_attributes.lookup(_character) == Attribute::Letter)) {
133         _token += _character;
134         _character = _source.read();
135     }
136
137     if (!_symbols.isMultiDelimiter(_token)) {
138         addError({
139             .message = LexicalError::InvalidMultiDelimiter(_token),
140             .row = startRow,
141             .column = startCol,
142         });
143
144         _token.clear();
145         return;
146     }
147
148     _code = _symbols.resolveMultiDelimiter(_token);
149
150     // Automata state: WRITE
151     addToken({
152         .code = _code,
153         .row = startRow,
154         .column = startCol,
155     });
156
157     _token.clear();
158 }
159
160 void Tokenizer::scanComment() {
161     size_t startRow = _source.cursor().row();
162     size_t startCol = _source.cursor().column();
163
164     _character = _source.read();
165
166     // Automata state: WRITE
167     if (_character != '*') {
168         addToken({
169             .code = static_cast<size_t>('('),
170             .row = startRow,
171             .column = startCol,
172         });
173
174         return;
175     }
176
177     _character = _source.read();
178
179     // Automata state: COMMENT
180     while (!_source.done()) {
181         // Automata state: COMMENT_CLOSE
182         if (_character == '*') {
183             _character = _source.read();
184
185             // Automata state: COMMENT_END
186             if (_character == ')') {
187                 _character = _source.read();
188
189                 return;
190             }
191         } else {
192             _character = _source.read();
193         }
194     }
195
196     // Automata state: ERROR
```

```
    addError({
        .message = LexicalError::UnclosedComment(),
        .row = startRow,
        .column = startCol,
    });
}

void Tokenizer::scanDelimiter() {
    addToken({
        .code = static_cast<size_t>(_character),
        .row = _source.cursor().row(),
        .column = _source.cursor().column(),
    });

    _character = _source.read();
}

void Tokenizer::scanInvalid() {
    addError({
        .message = LexicalError::InvalidCharacter(_character),
        .row = _source.cursor().row(),
        .column = _source.cursor().column(),
    });

    _character = _source.read();
}

const std::vector<Token>& Tokenizer::tokens() const {
    return _tokens;
}

void Tokenizer::addToken(const Token& token) {
    _tokens.push_back(token);
}

void Tokenizer::addError(const LexicalError& error) {
    _logger.message(error);
}
```

src/tokenizer/view/IdentifiersTableView/IdentifiersTableView.hpp

```cpp
#pragma once

#include <string>
#include <utility>
#include <vector>

#include <TableView.hpp>

using SymbolEntry = std::pair<std::string, size_t>;

class IdentifiersTableView {
private:
    TableView<SymbolEntry> _table;

public:
    IdentifiersTableView(
        std::string title,
        const std::vector<SymbolEntry>& data
    );

    void print() const;
};
```

src/tokenizer/view/IdentifiersTableView/IdentifiersTableView.cpp

```cpp
#include "IdentifiersTableView.hpp"

IdentifiersTableView::IdentifiersTableView(
    std::string title,
    const std::vector<SymbolEntry>& data
) :
    _table(std::move(title), data) {
    _table
        .addColumn(
            "Code", "{:>5}",
            [](const auto& pair) { return std::to_string(pair.second); }
        )
        .addColumn(
            "Value", "{:<30}",
            [](const auto& pair) { return pair.first; }
        );
}

void IdentifiersTableView::print() const {
    _table.print();
}
```

src/tokenizer/view/LiteralsTableView/LiteralsTableView.hpp

```cpp
#pragma once

#include <string>
#include <utility>
#include <vector>

#include <TableView.hpp>

using SymbolEntry = std::pair<std::string, size_t>;

class LiteralsTableView {
private:
    TableView<SymbolEntry> _table;

public:
    LiteralsTableView(
        std::string title,
        const std::vector<SymbolEntry>& data
    );

    void print() const;
};
```

src/tokenizer/view/LiteralsTableView/LiteralsTableView.cpp

```cpp
#include "LiteralsTableView.hpp"

LiteralsTableView::LiteralsTableView(
    std::string title,
    const std::vector<SymbolEntry>& data
) :
    _table(std::move(title), data) {
    _table
        .addColumn(
            "Code", "{:>5}",
            [](const auto& pair) { return std::to_string(pair.second); }
        )
        .addColumn(
            "Value", "{:<30}",
            [](const auto& pair) { return pair.first; }
        );
}

void LiteralsTableView::print() const {
    _table.print();
}
```

src/tokenizer/view/TokensTableView/TokensTableView.hpp

```cpp
#pragma once

#include <string>
#include <vector>

#include <SymbolStore.hpp>
#include <TableView.hpp>
#include <Tokenizer.hpp>

class TokensTableView {
private:
    TableView<Token> _table;

public:
    TokensTableView(
        std::string title,
        const SymbolStore& symbols,
        const std::vector<Token>& data
    );

    void print() const;
};
```

src/tokenizer/view/TokensTableView/TokensTableView.cpp

```cpp
#include "TokensTableView.hpp"

std::string getTypeLabel(SymbolType type) {
    switch (type) {
        case SymbolType::Ascii:
            return "Delimiter";
        case SymbolType::MultiDelimiter:
            return "Multi-Delimiter";
        case SymbolType::Keyword:
            return "Keyword";
        case SymbolType::Literal:
            return "Literal";
        case SymbolType::Identifier:
            return "Identifier";
        default:
            return "N/A";
    }
}

TokensTableView::TokensTableView(
    std::string title,
    const SymbolStore& symbols,
    const std::vector<Token>& data
) :
    _table(std::move(title), data) {
    _table
        .addColumn(
            "Code", "{:>5}",
            [](const auto& token) { return std::to_string(token.code); }
        )
        .addColumn(
            "Row", "{:>3}",
            [](const auto& token) { return std::to_string(token.row + 1); }
        )
        .addColumn(
            "Col", "{:>3}",
            [](const auto& token) { return std::to_string(token.column + 1); }
        )
        .addColumn(
            "Type", "{:<15}",
            [&symbols](const auto& token) {
                return getTypeLabel(symbols.lookupType(token.code));
            }
        )
        .addColumn("Value", "{:<30}", [&symbols](const auto& token) {
            return std::string(symbols.lookup(token.code));
        });
}

void TokensTableView::print() const {
    _table.print();
}
```

src/types/data/Type/Type.hpp

```hpp
#pragma once

enum class Type {
    ComplexInteger,
    ComplexFloat,
};
```

src/ui/ui.constants.hpp

```cpp
#pragma once

namespace Box {
constexpr auto Horizontal = "─";
constexpr auto Vertical = "│";
constexpr auto DownAndRight = "┌";
constexpr auto DownAndLeft = "┐";
constexpr auto UpAndRight = "└";
constexpr auto UpAndLeft = "┘";
constexpr auto VerticalAndRight = "├";
constexpr auto VerticalAndLeft = "┤";
constexpr auto DownAndHorizontal = "┬";
constexpr auto UpAndHorizontal = "┴";
constexpr auto VerticalAndHorizontal = "┼";
}  // namespace Box
```

src/ui/TableView/TableView.hpp

```hpp
#pragma once

#include <format>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

#include <ui.constants.hpp>

template <typename T>
class TableView {
private:
    struct Column {
        std::string name;
        std::string format;
        std::function<std::string(const T&)> extractor;
    };

    std::string _title;
    std::vector<T> _data;
    std::vector<Column> _columns;
    std::ostream& _out;

    [[nodiscard]] std::string border(
        const std::string& left,
        const std::string& junction,
        const std::string& right
    ) const;
    [[nodiscard]] std::string header() const;
    [[nodiscard]] std::string row(const T& item) const;
    [[nodiscard]] size_t columnWidth(const Column& col) const;
    static size_t parseWidth(const std::string& fmt);

public:
    TableView(
        std::string title,
        std::vector<T> data,
        std::ostream& out = std::cout
    );

    TableView& addColumn(
        const std::string& colName,
        const std::string& format,
        std::function<std::string(const T&)> extractor
    );

    void print() const;
};

#include "TableView.tpp"
```

src/ui/TableView/TableView.tpp

```cpp
template <typename T>
TableView<T>::TableView(
    std::string title,
    std::vector<T> data,
    std::ostream& out
) :
    _title(std::move(title)),
    _data(std::move(data)),
    _out(out) {}

template <typename T>
TableView<T>& TableView<T>::addColumn(
    const std::string& colName,
    const std::string& format,
    std::function<std::string(const T&)> extractor
) {
    _columns.push_back({colName, format, std::move(extractor)});

    return *this;
}

template <typename T>
void TableView<T>::print() const {
    if (!_title.empty()) {
        _out << _title << "\n";
    }

    _out << border(Box::DownAndRight, Box::DownAndHorizontal, Box::DownAndLeft) << "\n";
    _out << header() << "\n";
    _out << border(Box::VerticalAndRight, Box::VerticalAndHorizontal, Box::VerticalAndLeft) << "\

    for (const auto& item : _data) {
        _out << row(item) << "\n";
    }

    _out << border(Box::UpAndRight, Box::UpAndHorizontal, Box::UpAndLeft) << "\n";
}

template <typename T>
std::string TableView<T>::border(
    const std::string& left,
    const std::string& junction,
    const std::string& right
) const {
    std::string result = left;

    for (size_t i = 0; i < _columns.size(); ++i) {
        size_t width = columnWidth(_columns[i]);

        for (size_t j = 0; j < width + 2; ++j) {
            result += Box::Horizontal;
        }

        result += (i < _columns.size() - 1) ? junction : right;
    }

    return result;
}

template <typename T>
std::string TableView<T>::header() const {
    std::string result;
```

```cpp
    result += Box::Vertical;

    for (const auto& column : _columns) {
        result += std::format(" {:<{}} ", column.name, columnWidth(column));
        result += Box::Vertical;
    }

    return result;
}

template <typename T>
std::string TableView<T>::row(const T& item) const {
    std::string result;
    result += Box::Vertical;

    for (const auto& column : _columns) {
        std::string value = column.extractor(item);
        result += " " +
                  std::vformat(column.format, std::make_format_args(value)) +
                  " ";
        result += Box::Vertical;
    }

    return result;
}

template <typename T>
size_t TableView<T>::columnWidth(const Column& column) const {
    return std::max(parseWidth(column.format), column.name.length());
}

template <typename T>
size_t TableView<T>::parseWidth(const std::string& format) {
    size_t width = 0;

    for (char character : format) {
        if (character >= '0' && character <= '9') {
            width = (width * 10) + (character - '0');
        }
    }

    return width;
}
```

src/ui/TreeView/TreeView.hpp

```hpp
#pragma once

#include <functional>
#include <iostream>
#include <string>

#include <ui.constants.hpp>
#include <Tree.hpp>

template <typename T>
class TreeView {
private:
    std::string _title;
    const Tree<T>& _tree;
    std::ostream& _out;
    std::function<std::string(const T&)> _nodeFormatter;
    std::function<std::string(const T&)> _edgeFormatter;

    void printNode(
        const TreeNode<T>& node,
        const std::string& prefix,
        bool isLast
    ) const;

public:
    TreeView(std::string title, const Tree<T>& tree, std::ostream& out = std::cout);

    TreeView& setNodeFormatter(std::function<std::string(const T&)> formatter);
    TreeView& setEdgeFormatter(std::function<std::string(const T&)> formatter);

    void print() const;
};

#include "TreeView.tpp"
```

src/ui/TreeView/TreeView.tpp

```cpp
template <typename T>
TreeView<T>::TreeView(std::string title, const Tree<T>& tree, std::ostream& out) :
    _title(std::move(title)),
    _tree(tree),
    _out(out),
    _nodeFormatter(nullptr),
    _edgeFormatter(nullptr) {}

template <typename T>
TreeView<T>& TreeView<T>::setNodeFormatter(
    std::function<std::string(const T&)> formatter
) {
    _nodeFormatter = std::move(formatter);

    return *this;
}

template <typename T>
TreeView<T>& TreeView<T>::setEdgeFormatter(
    std::function<std::string(const T&)> formatter
) {
    _edgeFormatter = std::move(formatter);

    return *this;
}

template <typename T>
void TreeView<T>::print() const {
    auto& root = _tree.root();

    _out << _title << "\n";
    _out << _nodeFormatter(root.data()) << "\n";

    for (const auto& child : root.children()) {
        printNode(*child, "", child == root.children().back());
    }
}

template <typename T>
void TreeView<T>::printNode(
    const TreeNode<T>& node,
    const std::string& prefix,
    bool isLast
) const {
    std::string source = isLast
        ? Box::UpAndRight
        : Box::VerticalAndRight;

    std::string horizontalConnector = Box::Horizontal;

    std::string edgeLabel = _edgeFormatter(node.data());

    std::string nodeLabel = _nodeFormatter(node.data());

    std::string verticalConnector = isLast
        ? " "
        : Box::Vertical;

    _out << prefix << source << horizontalConnector;

    _out << edgeLabel << horizontalConnector;

```

```
63      _out << nodeLabel << "\n";
64
65      std::string childPrefix = prefix + verticalConnector + std::string(edgeLabel.size() + 2, ' ');
66
67      for (const auto& child : node.children()) {
68          printNode(*child, childPrefix, child == node.children().back());
69      }
70  }
```

src/utility/Args/Args.hpp

```hpp
#pragma once

#include <string>
#include <unordered_map>
#include <vector>

class Args {
private:
    struct StringArg {
        std::string shortName;
        bool required;
        std::string value;
        bool provided = false;
    };

    struct FlagArg {
        std::string shortName;
        bool value = false;
    };

    std::string _program;
    std::vector<std::string> _argv;

    std::unordered_map<std::string, StringArg> _strings;
    std::unordered_map<std::string, FlagArg> _flags;
    std::unordered_map<std::string, std::string> _aliases;

    void printUsage() const;

public:
    Args(int argc, char* argv[]);

    Args& expectString(
        const std::string& name,
        const std::string& shortName,
        bool required = false
    );
    Args& expectFlag(
        const std::string& name,
        const std::string& shortName
    );

    Args& parse();

    [[nodiscard]] const std::string& getString(const std::string& name) const;
    [[nodiscard]] bool getFlag(const std::string& name) const;
};
```

src/utility/Args/Args.cpp

```cpp
#include "Args.hpp"

#include <format>
#include <iostream>
#include <stdexcept>

Args::Args(int argc, char* argv[]) : _program(argv[0]) {
    for (int i = 1; i < argc; i++) {
        _argv.emplace_back(argv[i]);
    }
}

Args& Args::expectString(
    const std::string& name,
    const std::string& shortName,
    bool required
) {
    _strings[name] = {shortName, required, "", false};
    _aliases["--" + name] = name;
    _aliases["-" + shortName] = name;
    return *this;
}

Args& Args::expectFlag(
    const std::string& name,
    const std::string& shortName
) {
    _flags[name] = {shortName, false};
    _aliases["--" + name] = name;
    _aliases["-" + shortName] = name;
    return *this;
}

Args& Args::parse() {
    for (size_t i = 0; i < _argv.size(); i++) {
        const auto& arg = _argv[i];

        auto alias = _aliases.find(arg);

        if (alias == _aliases.end()) {
            std::cerr << "Unknown argument: " << arg << "\n";
            printUsage();
            exit(1);
        }

        const auto& name = alias->second;

        auto stringArg = _strings.find(name);
        if (stringArg != _strings.end()) {
            if (i + 1 >= _argv.size()) {
                std::cerr << "Missing value for: " << arg << "\n";
                printUsage();
                exit(1);
            }

            stringArg->second.value = _argv[++i];
            stringArg->second.provided = true;
            continue;
        }

        auto flagArg = _flags.find(name);
        if (flagArg != _flags.end()) {
```

```
            flagArg->second.value = true;
        }
    }

    for (const auto& [name, arg] : _strings) {
        if (arg.required && !arg.provided) {
            std::cerr << "Missing required argument: --" << name << "\n";
            printUsage();
            exit(1);
        }
    }

    return *this;
}

void Args::printUsage() const {
    std::cerr << "\nUsage: " << _program;

    for (const auto& [name, arg] : _strings) {
        if (arg.required) {
            std::cerr << " --" << name << " <value>";
        }
    }

    std::cerr << " [options]\n\nOptions:\n";

    for (const auto& [name, arg] : _strings) {
        std::cerr << std::format(
            "  --{}, -{} <value>{}\n", name, arg.shortName,
            arg.required ? " (required)" : ""
        );
    }

    for (const auto& [name, arg] : _flags) {
        std::cerr << std::format("  --{}, -{}\n", name, arg.shortName);
    }
}

const std::string& Args::getString(const std::string& name) const {
    auto it = _strings.find(name);

    if (it == _strings.end()) {
        throw std::runtime_error("Unknown string argument: " + name);
    }

    return it->second.value;
}

bool Args::getFlag(const std::string& name) const {
    auto it = _flags.find(name);

    if (it == _flags.end()) {
        throw std::runtime_error("Unknown flag argument: " + name);
    }

    return it->second.value;
}
```

src/utility/Logger/Logger.hpp

```hpp
#pragma once

#include <functional>
#include <iostream>
#include <string>
#include <vector>

template <typename T>
class Logger {
private:
    std::string _name;
    std::function<std::string(const T&)> _formatter;
    std::ostream& _out;
    std::vector<T> _messages;

public:
    Logger(
        std::string name,
        std::function<std::string(const T&)> formatter,
        std::ostream& out = std::cout
    );

    void message(const T& data);

    [[nodiscard]] const std::vector<T>& messages() const;
    void clear();
};

#include "Logger.tpp"
```

src/utility/Logger/Logger.tpp

```cpp
template <typename T>
Logger<T>::Logger(
    std::string name,
    std::function<std::string(const T&)> formatter,
    std::ostream& out
) :
    _name(std::move(name)), _formatter(std::move(formatter)), _out(out) {}

template <typename T>
void Logger<T>::message(const T& data) {
    _messages.push_back(data);
    _out << _name << " | " << _formatter(data) << "\n";
}

template <typename T>
const std::vector<T>& Logger<T>::messages() const {
    return _messages;
}

template <typename T>
void Logger<T>::clear() {
    _messages.clear();
}
```

src/utility/Queue/Queue.hpp

```hpp
#pragma once

#include <deque>
#include <functional>
#include <optional>
#include <vector>

template <typename T>
class Queue {
private:
    std::deque<T> _items;

public:
    Queue();
    Queue(const std::vector<T>& items);

    void push(const T& item);

    std::optional<T> pop();
    std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);

    void clear();

    std::optional<T> peek() const;
    bool isEmpty() const;
    size_t size() const;
};

#include "Queue.tpp"
```

src/utility/Queue/Queue.tpp

```cpp
template <typename T>
Queue<T>::Queue() : _items() {}

template <typename T>
Queue<T>::Queue(const std::vector<T>& items) :
    _items(items.begin(), items.end()) {}

template <typename T>
void Queue<T>::push(const T& item) {
    _items.push_back(item);
}

template <typename T>
std::optional<T> Queue<T>::pop() {
    if (_items.empty()) {
        return std::nullopt;
    }

    T item = std::move(_items.front());
    _items.pop_front();

    return item;
}

template <typename T>
std::optional<T> Queue<T>::popUntil(
    const std::function<bool(const T&)>& predicate
) {
    while (!_items.empty()) {
        if (predicate(_items.front())) {
            return _items.front();
        }

        _items.pop_front();
    }

    return std::nullopt;
}

template <typename T>
void Queue<T>::clear() {
    _items.clear();
}

template <typename T>
std::optional<T> Queue<T>::peek() const {
    if (_items.empty()) {
        return std::nullopt;
    }

    return _items.front();
}

template <typename T>
bool Queue<T>::isEmpty() const {
    return _items.empty();
}

template <typename T>
size_t Queue<T>::size() const {
    return _items.size();
}
```

src/utility/Stack/Stack.hpp

```hpp
#pragma once

#include <vector>
#include <optional>
#include <functional>

template <typename T>
class Stack {
private:
  std::vector<T> _items;

public:
    Stack();
    Stack(const std::vector<T>& items);

    void push(const T& item);

    std::optional<T> pop();
    std::optional<T> popUntil(const std::function<bool(const T&)>& predicate);

    void clear();

    std::optional<T> peek() const;
    bool isEmpty() const;
    size_t size() const;
};

#include "Stack.tpp"
```

src/utility/Stack/Stack.tpp

```cpp
template <typename T>
Stack<T>::Stack() : _items() {}

template <typename T>
Stack<T>::Stack(const std::vector<T>& items) : _items(items) {}

template <typename T>
void Stack<T>::push(const T& item) {
    _items.push_back(item);
}

template <typename T>
std::optional<T> Stack<T>::pop() {
    if (_items.empty()) {
        return std::nullopt;
    }

    T item = std::move(_items.back());
    _items.pop_back();

    return item;
}

template <typename T>
std::optional<T> Stack<T>::popUntil(const std::function<bool(const T&)>& predicate) {
    while (!_items.empty()) {
        if (predicate(_items.back())) {
            return _items.back();
        }

        _items.pop_back();
    }

    return std::nullopt;
}

template <typename T>
void Stack<T>::clear() {
    _items.clear();
}

template <typename T>
std::optional<T> Stack<T>::peek() const {
    if (_items.empty()) {
        return std::nullopt;
    }

    return _items.back();
}

template <typename T>
bool Stack<T>::isEmpty() const {
    return _items.empty();
}

template <typename T>
size_t Stack<T>::size() const {
    return _items.size();
}
```

src/utility/Tree/Tree.hpp

```cpp
#pragma once

#include <TreeNode.hpp>

template <typename T>
class Tree {
private:
    std::unique_ptr<TreeNode<T>> _root;

public:
    Tree(T data);

    TreeNode<T>& root();
    const TreeNode<T>& root() const;
};

#include "Tree.tpp"
```

src/utility/Tree/Tree.tpp

```cpp
template <typename T>
Tree<T>::Tree(T data) : _root(std::make_unique<TreeNode<T>>(data)) {}

template <typename T>
TreeNode<T>& Tree<T>::root() {
    return *_root;
}

template <typename T>
const TreeNode<T>& Tree<T>::root() const {
    return *_root;
}
```

src/utility/Tree/Node/TreeNode.hpp

```hpp
#pragma once

#include <vector>
#include <memory>

template <typename T>
class TreeNode {
private:
    T _data;
    std::vector<std::unique_ptr<TreeNode<T>>> _children;

public:
    TreeNode(T data);
    TreeNode(const TreeNode&) = delete;
    TreeNode& operator=(const TreeNode&) = delete;
    TreeNode(TreeNode&&) = default;
    TreeNode& operator=(TreeNode&&) = default;

    const T& data() const;
    T& data();

    const std::vector<std::unique_ptr<TreeNode<T>>>& children() const;
    TreeNode<T>& grow(T data);
    void prune();
};

#include "TreeNode.tpp"
```

src/utility/Tree/Node/TreeNode.tpp

```cpp
template <typename T>
TreeNode<T>::TreeNode(T data) : _data(data) {}

template <typename T>
const T& TreeNode<T>::data() const {
    return _data;
}

template <typename T>
T& TreeNode<T>::data() {
    return _data;
}

template <typename T>
const std::vector<std::unique_ptr<TreeNode<T>>>& TreeNode<T>::children() const {
    return _children;
}

template <typename T>
TreeNode<T>& TreeNode<T>::grow(T data) {
    _children.push_back(std::make_unique<TreeNode<T>>(data));
    return *_children.back();
}

template <typename T>
void TreeNode<T>::prune() {
    _children.clear();
}
```